Convolutional Neural Networks

Locality, Weight Sharing, and Hierarchy in Images

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Why MLPs struggle with images

Where we are

So far every network was a **fully-connected** MLP on a flat vector $x \in \mathbb{R}^d$.

$$h = \varphi(Wx + b), \qquad \mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$$

That worked for tabular data. But an **image** is not an unstructured vector — it has **spatial structure** the MLP throws away.

An image is a tensor

A colour image is a 3-D array — height, width, and colour channels:

$$X \in \mathbb{R}^{H \times W \times C}$$

- $H \times W$ — a **grid** of pixels with a real **geometry**;
- C — channels (e.g. $C = 3$ for RGB);
- neighbouring pixels are **strongly correlated**; far-apart ones usually are not.

a 224×224 RGB image is $224 \times 224 \times 3 = 150{,}528$ numbers

Flattening destroys structure V

To feed an MLP we must **flatten** X into a vector:

$$X \in \mathbb{R}^{H \times W \times C} \quad \xrightarrow{\text{flatten}} \quad x \in \mathbb{R}^{HWC}$$

Flattening discards **which pixels are neighbours**. Pixel (i, j) and $(i, j+1)$ — adjacent in the image — become two arbitrary, unrelated entries of x.

the MLP would have to **re-learn** the grid geometry from scratch, from data

Parameter explosion D

Connect a flattened $224 \times 224 \times 3$ image to a modest hidden layer of 1000 units:

$$\underbrace{150{,}528}_{\text{inputs}} \times \underbrace{1000}_{\text{hidden units}} = 1.5 \times 10^8 \quad \text{weights}$$

One layer — 150 million parameters. Deeper nets become impossible: too many weights to store, and far too many to fit without massive overfitting.

and this is before we add a single second layer

Three priors images obey

Images are **not** arbitrary vectors. Three structural facts we should build in:

1. **Locality** — meaningful patterns (edges, corners) are **local**: a pixel is explained by its **neighbours**, not by pixels across the image.
2. **Translation equivariance** — a cat is a cat in the **top-left** or the **bottom-right**; the **same** detector should apply everywhere.
3. **Hierarchy** — edges compose into textures, textures into parts, parts into objects.

The CNN answer

Images have **spatial structure**.
CNNs exploit **locality**, **weight sharing**, and **hierarchy**.

Image prior	**CNN mechanism**
locality	small **local** kernels, not full connections
translation equivariance	**share** one kernel across all positions
hierarchy	**stack** conv layers → growing features

Convolution

1-D convolution: a sliding weighted sum

Slide a small **kernel** K of weights over a signal x, taking a weighted sum at each spot:

$$y[i] = \sum_{u} K[u]\, x[i+u]$$

- the **same** weights K are reused at **every** position i — that is **weight sharing**;
- each output looks at only a **few** neighbouring inputs — that is **locality**.

a 3-tap kernel $[-1, 0, 1]$ computes a local **difference** — a 1-D edge detector

2-D convolution

For an image, slide a 2-D kernel over both axes:

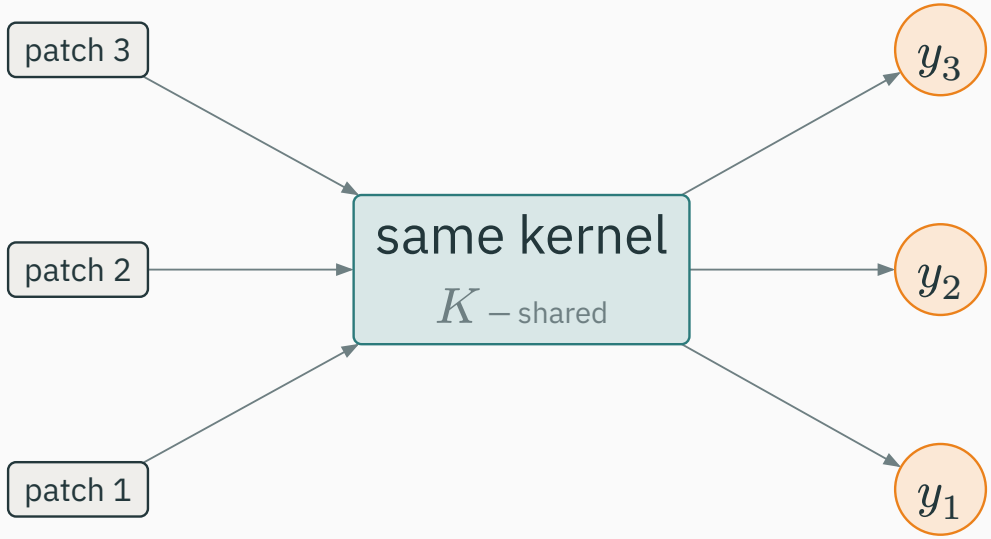
$$Y[i, j] = \sum_{u,v} K[u, v]\, X[i + u, j + v]$$

- K is a small window (e.g. 3×3) of **learned** weights;
- one output pixel is a weighted sum of a small **patch** of the input;
- the **same** K produces **every** output pixel.

few weights, reused everywhere — the opposite of a fully-connected layer

Weight sharing, visually

The **same** kernel K is applied at **every** spatial location — not a fresh weight set per position:



one small set of weights serves the whole image — this is what makes convolution cheap and translation-aware

Worked example: one conv step D

Kernel = horizontal-edge detector; patch = top-left 3×3 of the input:

$$K = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix}, \quad X_{\text{patch}} = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}$$

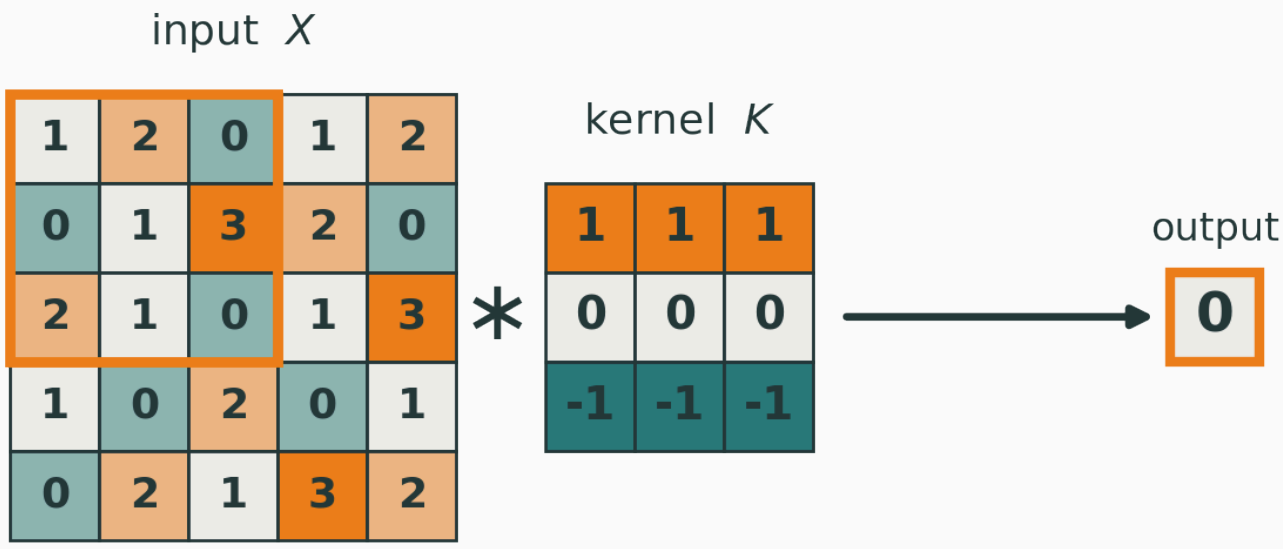
Elementwise-multiply and sum — take it one row at a time:

- top row: $1 \cdot 1 + 1 \cdot 2 + 1 \cdot 0 = 3$;
- middle row: $0 \cdot 0 + 0 \cdot 1 + 0 \cdot 3 = 0$ — the zeros erase it;
- bottom row: $(-1) \cdot 2 + (-1) \cdot 1 + (-1) \cdot 0 = -3$.

$$Y = 3 + 0 + (-3) = 0$$

$Y = 0$ — top brightness $=$ bottom brightness, so no horizontal edge here

Convolution, visually



$$(1+2+0)\cdot 1 + (0+1+3)\cdot 0 + (2+1+0)\cdot(-1)$$

$$= 3 - 3 = 0$$

top row minus bottom row: this kernel **fires** only where brightness changes vertically

A kernel is a feature detector

Each kernel **responds** to one local pattern — it lights up where that pattern appears:

Kernel	**Detects**
$[-1, 0, 1]$ (horizontal)	vertical edges
$[1, 1, 1; 0, 0, 0; -1, -1, -1]$	horizontal edges
centre-surround	blobs / corners
oriented bars	textures at an angle

In a CNN these kernels are **not** hand-designed — they are **learned** by gradient descent.

Cross-correlation vs convolution optional

True mathematical convolution **flips** the kernel before sliding:

$$(K * X)[i, j] = \sum_{u,v} K[u, v]\, X[i - u, j - v]$$

Deep-learning libraries skip the flip — they compute **cross-correlation**:

$$Y[i, j] = \sum_{u,v} K[u, v]\, X[i + u, j + v]$$

Since K is **learned**, the flip is irrelevant — the network just learns the flipped kernel. Everyone calls it “convolution” anyway. We will too.

Multiple kernels → channels

One kernel gives one **feature map**. Use C_{out} kernels to detect C_{out} patterns:

Tensor	Shape	Meaning
input X	$H \times W \times C_{\text{in}}$	C_{in} input channels
weights W	$k_h \times k_w \times C_{\text{in}} \times C_{\text{out}}$	C_{out} kernels
output Y	$H' \times W' \times C_{\text{out}}$	C_{out} feature maps

each output channel sums over **all** input channels, then across the $k_h \times k_w$ window

Parameter count of a conv layer D

Each of the C_{out} kernels has $k_h \cdot k_w \cdot C_{\text{in}}$ weights, plus one bias:

$$\text{params} = k_h \cdot k_w \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$$

Worked example — a 3×3 conv, $C_{\text{in}} = 64$, $C_{\text{out}} = 128$:

$$3 \cdot 3 \cdot 64 = 576 \quad \text{weights in one kernel}$$

$$576 \cdot 128 + 128 = 73{,}728 + 128 = 73{,}856$$

Independent of image size $H \times W$ — a 3×3 kernel costs the same on a 32×32 or a 2000×2000 image. Contrast the MLP's 1.5×10^8.

Interactive: convolution playground I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/convolution-visualizer

Paint an input grid, pick a kernel, and watch the window slide — see each multiply-add produce one output pixel, and how the kernel choice changes the feature map.

Q. What kernel would detect a $45°$ diagonal edge?

Padding, stride, and output size

Convolution shrinks the image

A $k \times k$ kernel cannot centre on the border pixels, so the output is **smaller**:

$$H' = H - k + 1 \quad \text{(valid convolution, no padding)}$$

- a 3×3 kernel on $32 \times 32 \to 30 \times 30$;
- stack many layers and the map **shrinks away**.

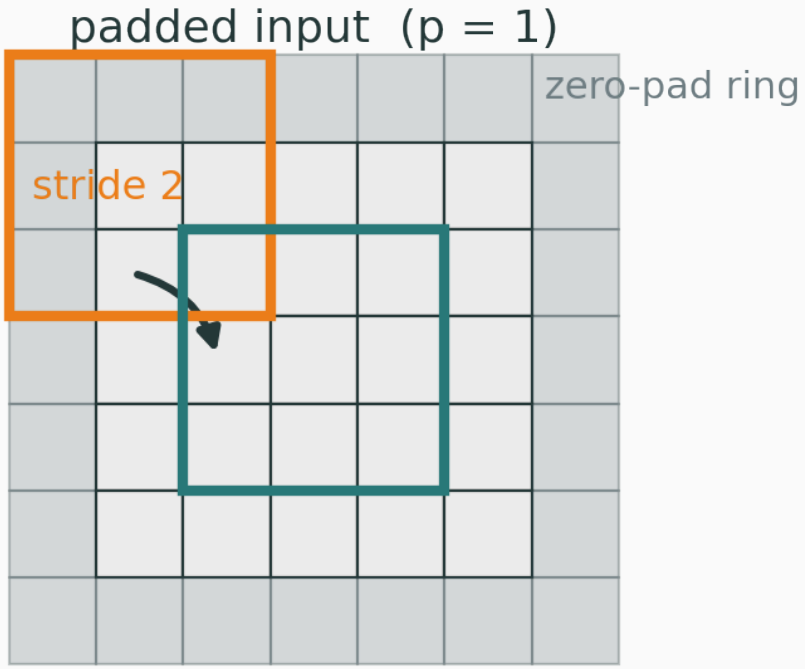
fix: pad the border with zeros

Padding

Add a ring of p zeros around the input before convolving:

- $p = 0$ — **valid**: no padding, output shrinks;
- $p = \lfloor k/2 \rfloor$ — **same**-ish: output keeps the input size (for stride 1);
- e.g. $k = 3, p = 1$ or $k = 5, p = 2$ preserve $H \times W$.

Padding



Stride

Stride s = how far the kernel jumps between outputs. $s > 1$ **downsamples**:

- $s = 1$ — evaluate at every position (dense);
- $s = 2$ — skip every other position → output roughly **halved** in each dimension.

stride is a cheap way to shrink the spatial size **inside** the convolution

The output-size formula D

Padding p, kernel k, and stride s combine into one count:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

Same — $H = 32, k = 5, p = 2, s = 1$:

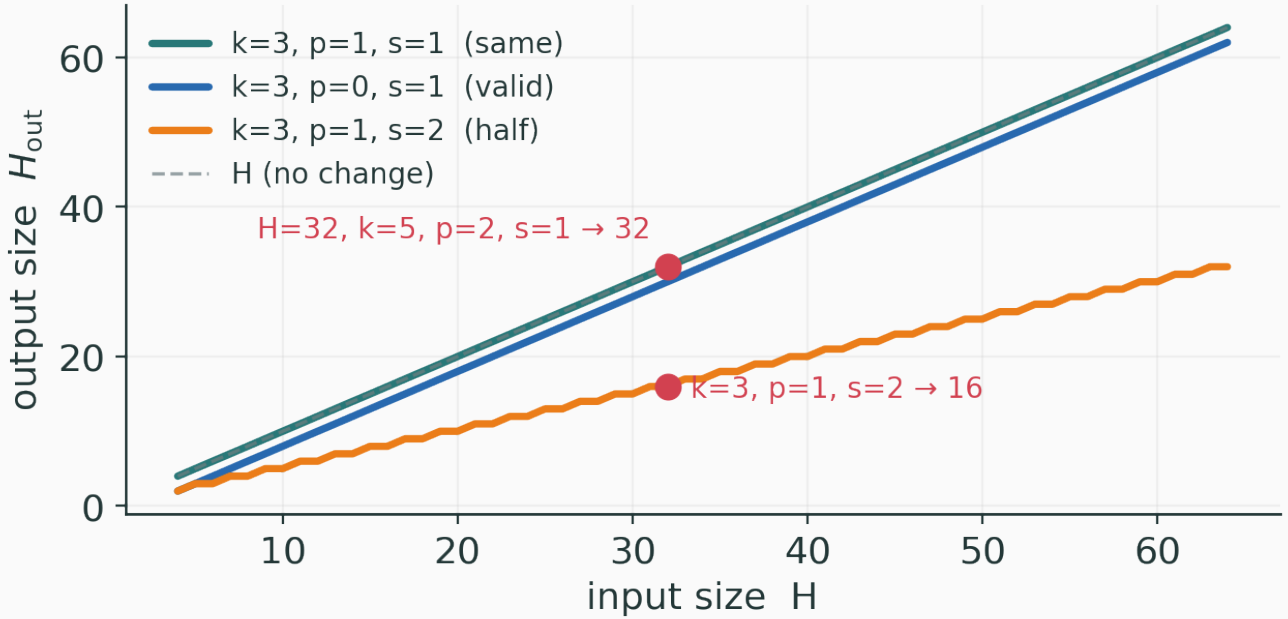
$$\left\lfloor \frac{32 + 4 - 5}{1} \right\rfloor + 1 = 31 + 1 = 32$$

Halved — $H = 32, k = 3, p = 1, s = 2$:

$$\left\lfloor \frac{32 + 2 - 3}{2} \right\rfloor + 1 = 15 + 1 = 16$$

check: a 7×7 kernel, $s = 1$ — what padding keeps $H \times W$? ($p = 3$, since $\lfloor k/2 \rfloor = 3$)

Output size at a glance v



$p = \lfloor k/2 \rfloor$, $s = 1$ keeps size (same) · $s = 2$ halves it · no padding slowly shrinks

Interactive: padding & stride I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/convolution-visualizer

Sweep padding and stride and watch the output grid grow, hold, or shrink — and confirm the $\lfloor (H + 2p - k)/s \rfloor + 1$ count.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/padding-stride

a dedicated dial for p and s with the live output-size readout.

Equivariance and pooling

Translation equivariance D

Because one kernel is **shared** across all positions, shifting the input **shifts the output the same way**:

$$f(T_\Delta\, x) = T_\Delta\, f(x)$$

where T_Δ is a shift by Δ. Move the cat right by 10 pixels → its feature map moves right by 10 pixels.

This is **equivariance**, not **invariance**. A conv layer does not **ignore** position — it **tracks** it. Invariance ("cat anywhere → same label") is built **later**, by pooling and the final head.

Pooling: summarise a neighbourhood

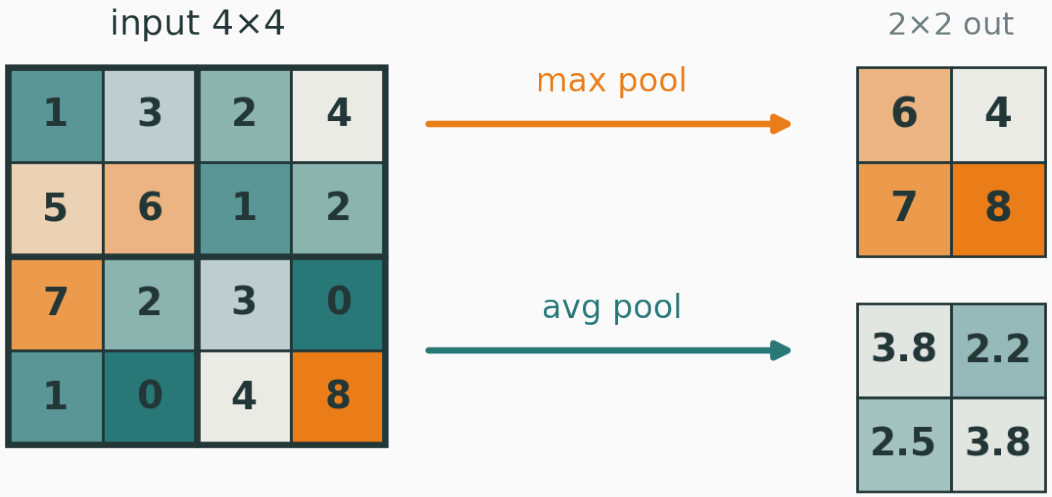
Slide a window over each feature map and replace it by a **single summary** — no learned weights:

$$y = \max_{(i,j) \in R} x_{ij} \qquad \text{(max pooling)}$$

$$y = \frac{1}{|R|} \sum_{(i,j) \in R} x_{ij} \qquad \text{(average pooling)}$$

typically a 2×2 window with stride 2 → halves H and W

Pooling, worked V



max keeps the **strongest** response in each region; average keeps the **mean**

What pooling buys us

- **Downsampling** — fewer spatial positions → less compute and memory downstream;
- **Larger receptive field** — each later neuron now sees more of the input;
- **Local robustness** — a small shift within a window leaves $\max$ unchanged → a step toward **invariance**;
- but **information loss** — pooling is not invertible; you discard **where** inside the window.

Global average pooling

At the very end, collapse each feature map to a **single number** — its channel-wise mean:

$$h_c = \frac{1}{HW} \sum_{i,j} X_{ijc}$$

- turns an $H \times W \times C$ map into a length-C vector;
- no parameters, and works at **any** input resolution;
- the modern replacement for a giant flatten-then-dense head.

GAP is where a stack of **equivariant** conv layers finally becomes **invariant** to position

Interactive: pooling I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/pooling-visualizer

drag a window over a feature map and compare max vs average pooling, and watch a small shift of the input leave the max output unchanged.

Receptive field

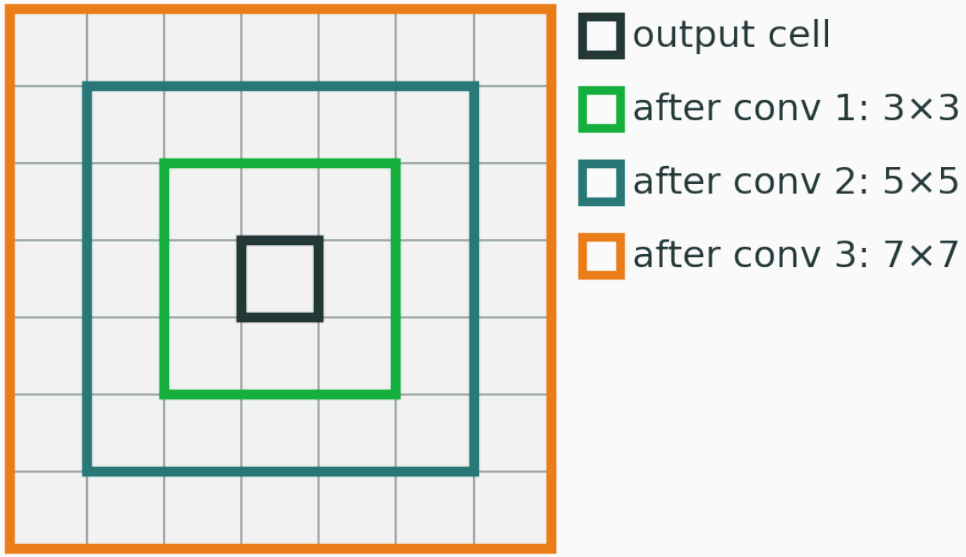
What one neuron sees

The **receptive field** of an output unit = the region of the **input** that can affect it.

- a single 3×3 conv → receptive field 3×3;
- stack a **second** 3×3 conv → each new unit sees a 3×3 patch of units that **each** saw 3×3 → an effective 5×5 input region.

depth grows the receptive field — deeper neurons see more context

Receptive field grows with depth



stacking 3×3 convs: $r_\ell = r_{\ell-1} + (k-1)$

three stacked 3×3 convs: receptive field $3 \to 5 \to 7$

Two small kernels beat one big one D

Two stacked 3×3 convs cover the same 5×5 field as one 5×5 conv — but cheaper:

	two 3×3	**one 5×5**
receptive field	5×5	5×5
parameters	$2 \cdot 3 \cdot 3C^2 = 18C^2$	$5 \cdot 5C^2 = 25C^2$
nonlinearities	2	1

Fewer parameters **and** more nonlinearity → why modern nets stack many small 3×3 kernels.

Receptive field recursion D

For stride-1 layers, the receptive field grows by $k_\ell - 1$ at each layer:

$$r_\ell = r_{\ell-1} + (k_\ell - 1)$$

Worked example — three 3×3 layers, starting from $r_0 = 1$:

- after layer 1: $r_1 = 1 + (3 - 1) = 3$;
- after layer 2: $r_2 = 3 + (3 - 1) = 5$;
- after layer 3: $r_3 = 5 + (3 - 1) = 7$.

strided or pooled layers grow it **much** faster — the receptive field can cover the whole image within a few blocks

Interactive: receptive-field grower I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/receptive-field-grower

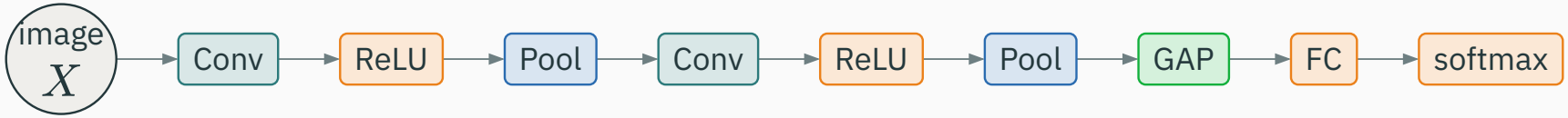
Add conv layers one at a time and watch the highlighted receptive field expand back through the network onto the input image.

Q. How many 3×3 stride-1 layers give a receptive field of at least 15×15?

The CNN architecture

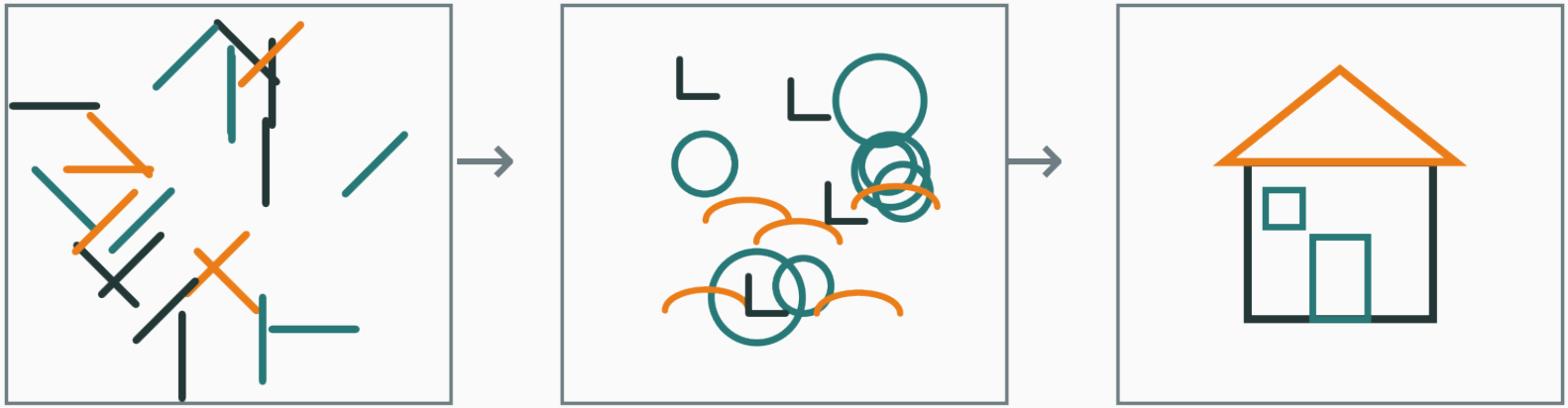
The typical pattern v

A CNN interleaves **convolution**, **nonlinearity**, and **downsampling**, then classifies:



spatial size **shrinks**, channel count **grows** — trading resolution for semantics

A feature hierarchy V



early: edges

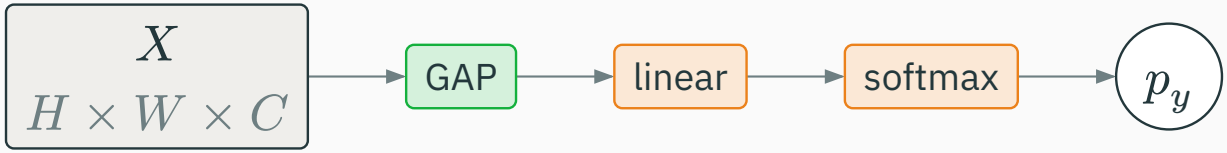
middle: textures / parts

late: objects

early layers → edges · middle → textures & parts · late → objects & semantics

The classification head

Convolutions produce features; a small head turns them into class probabilities:



$$p = \mathrm{softmax}(Wh + b), \qquad \mathcal{L} = -\log p_y$$

same cross-entropy loss as an MLP — only the **feature extractor** changed

Worked example: a tiny CNN D

Track the **shapes** through a small classifier on a 32×32 RGB image:

Layer	Output shape	Note
input	$32 \times 32 \times 3$	RGB image
conv 3×3, 16, $p = 1$	$32 \times 32 \times 16$	same padding
max-pool 2×2, $s = 2$	$16 \times 16 \times 16$	halve spatial
conv 3×3, 32, $p = 1$	$16 \times 16 \times 32$	more channels
global avg pool	32	collapse H, W
linear $\to$ 10	10	class scores

Tiny CNN: parameter count D

$$\text{conv1} = 3 \cdot 3 \cdot 3 \cdot 16 + 16 = 448$$

$$\text{conv2} = 3 \cdot 3 \cdot 16 \cdot 32 + 32 = 4640$$

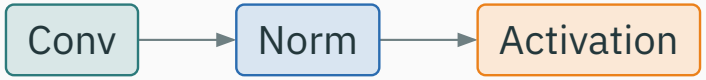
$$\text{linear} = 32 \cdot 10 + 10 = 330$$

total $\approx 448 + 4640 + 330 = 5418$ **parameters**

a full-image MLP needed 10^8 — the CNN does more with **four orders of magnitude** fewer weights

The modern conv block

In practice each “conv” step is a small **block**:



Normalization (L6) stabilises training; **residual connections** (L6) let gradients skip layers. The same ideas that tamed deep MLPs return — deep CNNs need them just as much.

Historical anchors optional

- **LeNet** (1990s) — small CNNs read handwritten **digits** on cheques and mail.
- **AlexNet** (2012) — a deep CNN trained on GPUs cut ImageNet error sharply, and kicked off the modern deep-learning wave.

The **ingredients** — conv, pooling, ReLU, depth — are what matter here; the exact architectures are landmarks, not laws. (dates and details are approximate)

Backprop through convolution

Convolution is a linear operator

For fixed weights, convolution is **linear** in the input:

$$y = K * x$$

Linear $\Longrightarrow$ it has a well-defined **transpose**, and backprop is **automatic**. No new machinery — the same chain rule from L4 applies.

The two gradients D

Given the upstream gradient $\partial \mathcal{L} / \partial y$, backprop needs two things:

- **gradient w.r.t. the kernel** — **correlate** the input with the output gradient (each weight is reused everywhere, so its gradient **sums** over all positions);
- **gradient w.r.t. the input** — **spread** the output gradient back through the kernel — a convolution with the **flipped** kernel.

weight sharing in the forward pass ⟹ gradient **summing** in the backward pass

Conv as matrix multiply optional

Convolution can be **unrolled** into a single matrix multiply (`im2col`):

$$y = K_{\text{mat}}\, x$$

- lay each input patch out as a column, stack the kernel into a matrix;
- convolution becomes one big matmul — exactly what GPUs are fastest at.

This is how libraries **implement** conv efficiently. The gradient is then just the transpose $K_{\text{mat}}^{\top}$ — standard linear-layer backprop.

Training CNNs in practice

Data augmentation

Images admit **label-preserving** transforms — a flipped cat is still a cat:

- random **crop** and **resize**; horizontal **flip**;
- **colour jitter** (brightness, contrast); small **rotations**;
- then **normalize** to a standard range.

Augmentation multiplies your effective dataset and bakes in invariances — one of the cheapest, most effective regularisers in vision.

Input normalization

Standardise each channel using the **training-set** mean and standard deviation:

$$x' = \frac{x - \mu}{\sigma}$$

- keeps inputs on a consistent scale → healthier gradients, faster training;
- compute μ, σ on the **train** set only, then reuse them for val / test.

Transfer learning

Rarely train from scratch — start from a network **pretrained** on a large dataset:

- keep the pretrained **backbone** (it already knows edges, textures, parts);
- replace the **head** with one for your classes;
- **fine-tune** — train the head, optionally unfreeze the backbone with a small LR.

strong results from little data — reuse features, don't relearn them

Common failure modes

Symptom	Usual cause
shape mismatch error	wrong tensor shape / forgot a dimension
colours look wrong	channel order (RGB vs BGR, NCHW vs NHWC)
tiny feature map, weak acc	too much downsampling too early
overfits fast	no data augmentation
great train, poor test	`train()` / `eval()` mode bug (norm, dropout)

Summary

Ingredient	What it buys
convolution	local feature extraction, weight sharing
multiple kernels	many features → channels
padding / stride	control output resolution
pooling	downsample + local robustness
receptive field	how much context a neuron sees
depth	edges → textures → parts → objects

Final mental model — one idea

An MLP asks **every pixel about every other**.
A CNN asks the **right question**:
what local patterns appear, and where?

locality + weight sharing + hierarchy = vision that scales

Classification tells us **what** is in the image.

Next: **where** is it, and **which pixels** belong to it? — detection and segmentation.